

Motion Detection Application: System Architecture

Version 1.0 · Windows desktop · Python, OpenCV, Tkinter · fully offline

1. Purpose and scope

The application reads the client's webcam, shows the live feed, and displays a "Motion Detected!" overlay on the feed when movement occurs. Nothing is recorded, saved or sent anywhere. All processing happens locally and in memory.

In scope: FR-01 to FR-10 and the listed non-functional requirements. Out of scope for v1: recording, remote notifications, multiple cameras, user accounts.

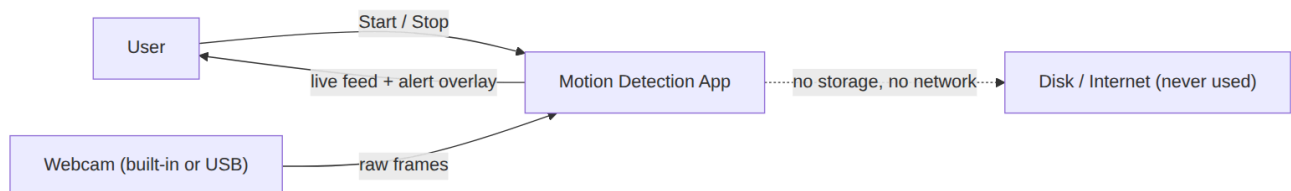
2. Architectural style

A single-process desktop application with a producer/consumer design:

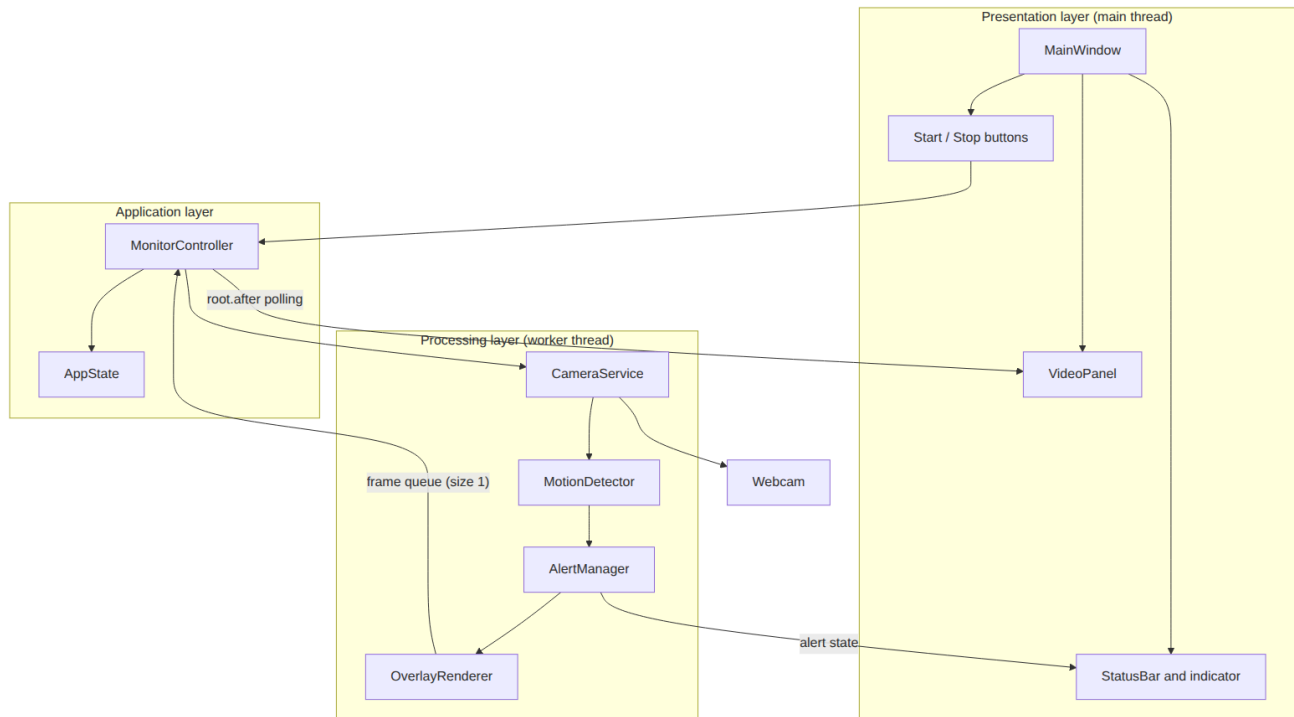
- A **capture worker thread** produces frames and detection results.
- The **Tkinter main thread** consumes them and renders the interface.
- A **bounded queue of size 1** connects the two, so the UI always shows the newest frame and never lags behind.

Tkinter is not thread-safe, so only the main thread touches widgets. The worker never calls UI code directly.

3. System context



4. Component architecture

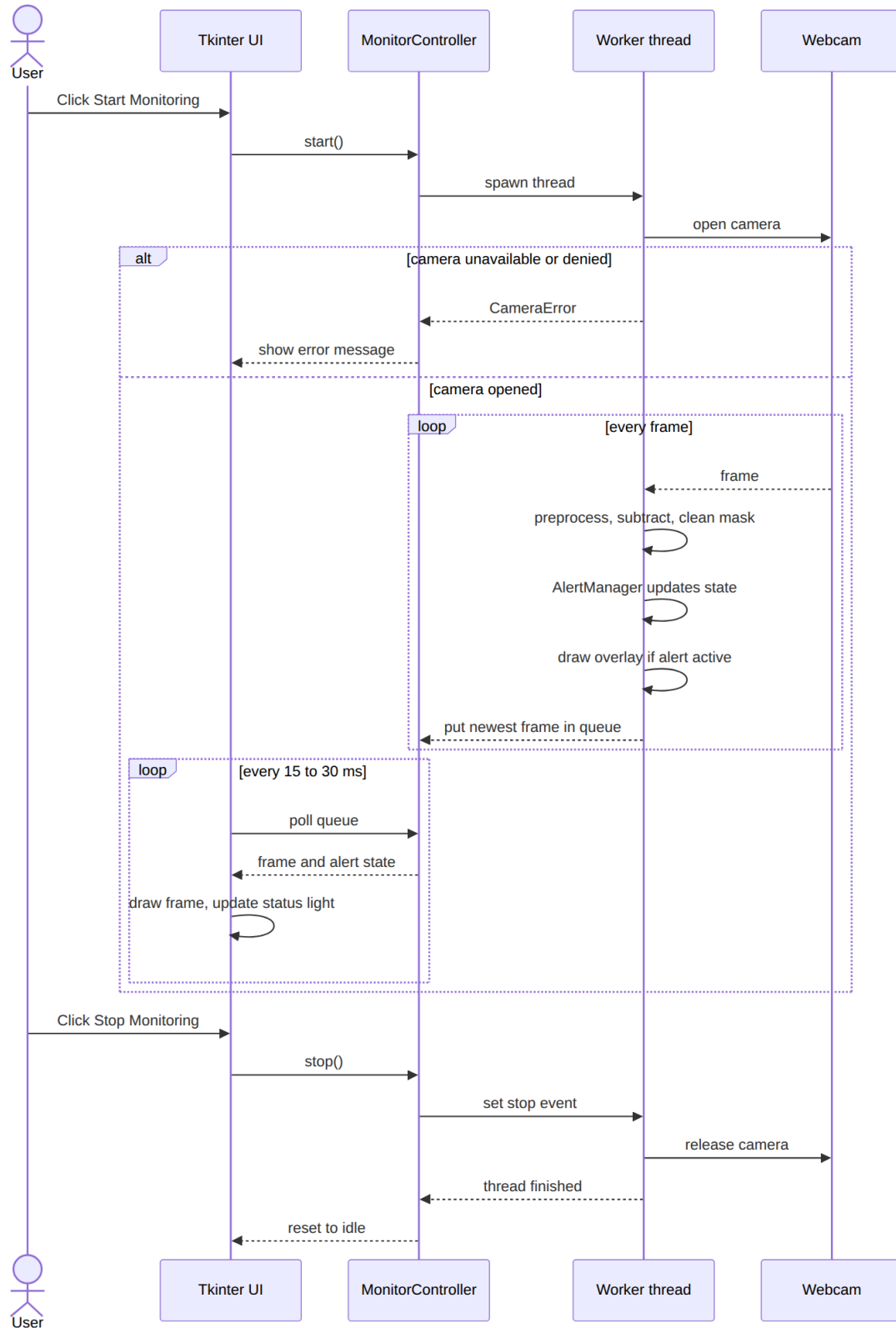


4.1 Component responsibilities

Component	Layer	Responsibility
MainWindow	Presentation	Builds the window, lays out widgets, handles window close (always releases the camera).
VideoPanel	Presentation	Converts the BGR frame to RGB, then to <code>PhotoImage</code> , and draws it on a Tkinter label.
StatusBar	Presentation	Shows "No Motion Detected", "Motion Detected!" or an error message, with a green or red indicator.
Start / Stop buttons	Presentation	Call the controller. Enabled or disabled according to app state to prevent double starts.
MonitorController	Application	Owns the worker thread, the stop event and the frame queue. Runs the UI polling loop. Single entry point for start and stop.
AppState	Application	Holds the current state (see section 7) and notifies the UI of changes.
CameraService	Processing	Opens, reads and releases <code>cv2.VideoCapture</code> . Detects open and read failures and raises typed errors.
MotionDetector	Processing	Preprocesses frames, runs background subtraction, returns a motion score and bounding boxes.
AlertManager	Processing	Turns raw per-frame detections into a stable alert state using confirmation frames and a cooldown.
OverlayRenderer	Processing	Draws the "Motion Detected!" banner and optional boxes onto the frame in memory.

5. Runtime behaviour

5.1 Start-to-alert sequence



5.2 Threading rules

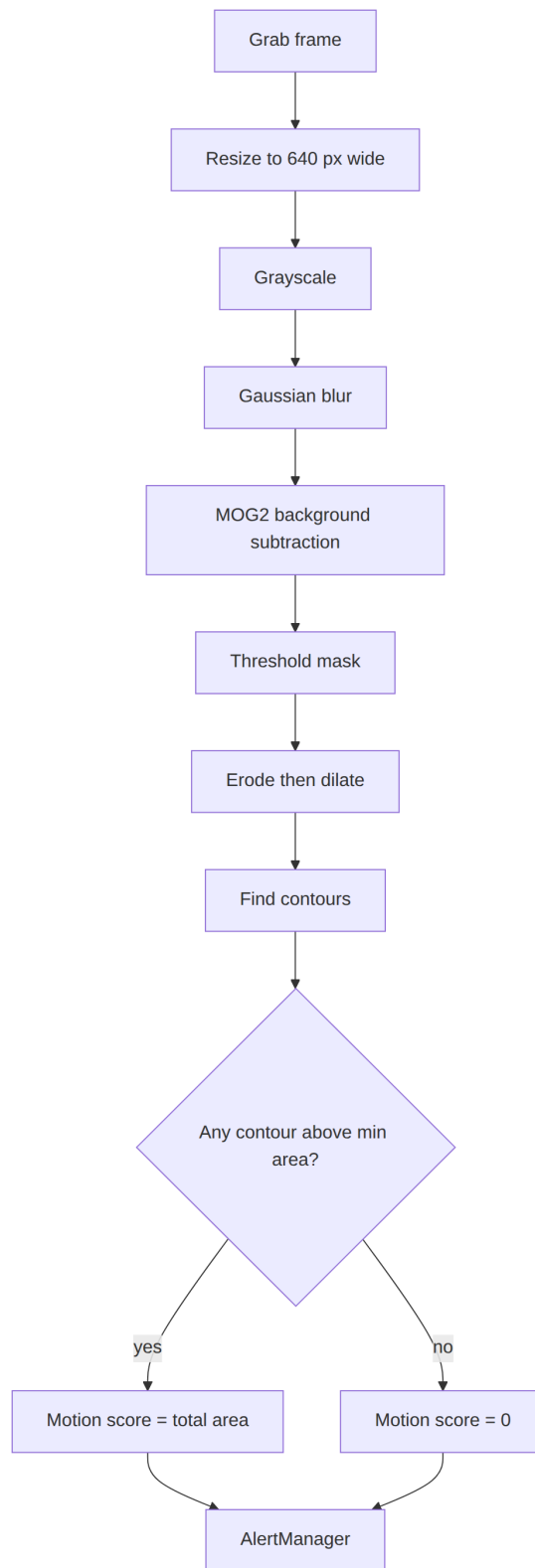
1. The worker thread owns the camera. No other thread reads from it.
2. The queue has `maxsize=1`. When it is full, the worker discards the old frame and puts the new one in. This drops stale frames rather than letting latency build up.
3. Stopping uses a `threading.Event`. The worker checks it every loop, exits cleanly, and releases the camera in a `finally` block.
4. The worker is a daemon thread, so a crash of the UI cannot leave the process hanging.

Core pattern:

```
# worker thread
while not stop_event.is_set():
    ok, frame = camera.read()
    if not ok:
        raise CameraError("Lost camera feed")
    result = detector.process(frame)
    state = alert_manager.update(result)
    frame = overlay.draw(frame, state, result)
    push_latest(queue, (frame, state))

# main thread
def poll():
    try:
        frame, state = queue.get_nowait()
        video_panel.show(frame)
        status_bar.set(state)
    except Empty:
        pass
    root.after(15, poll)
```

6. Detection pipeline



6.1 Why background subtraction

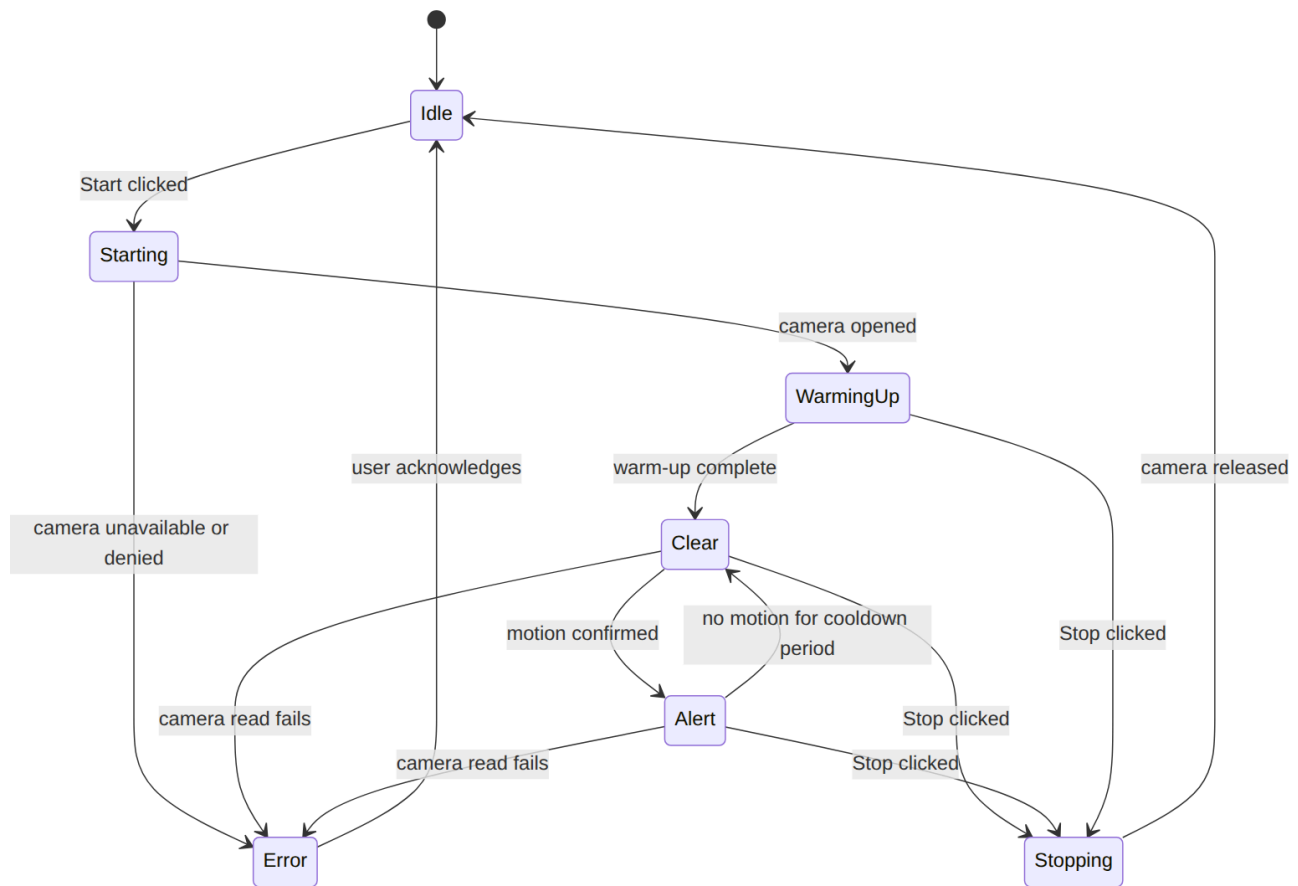
Frame differencing is simpler but misses slow movement and flags every lighting flicker. MOG2 models the background over time, adapts to gradual lighting changes, and can flag shadows so they can be ignored. It is built into OpenCV and fast enough for real time on ordinary laptops.

6.2 Starting parameters (all configurable)

Parameter	Starting value	Effect
Processing width	640 px	Lower means faster, less sensitive to small motion.
Blur kernel	21 x 21	Larger suppresses more sensor noise.
MOG2 history	500 frames	How long the background model remembers.
MOG2 varThreshold	25	Higher means fewer false positives.
Shadow handling	On, shadow pixels removed by threshold at 200	Stops shadows counting as movement.
Morphology kernel	5 x 5 ellipse	Removes speckles and fills small holes.
Min contour area	about 1,500 px at 640 wide	Ignores tiny movement such as dust or flicker.
Confirmation frames	3 consecutive	Requires sustained movement before alerting.
Cooldown	2 seconds	Keeps the alert visible after movement stops.
Warm-up	30 frames	Detections ignored while the background model learns.

These are the main values to adjust once the client sees it in their real room.

7. State model



State	Status text	Indicator	Buttons
Idle	Monitoring stopped	Grey	Start enabled
Starting / WarmingUp	Starting camera...	Grey	Stop enabled
Clear	No Motion Detected	Green	Stop enabled
Alert	Motion Detected! (also on overlay)	Red	Stop enabled
Error	Specific error message	Grey	Start enabled

8. Alert logic

The AlertManager is a small state machine so that the alert is stable:

1. Each frame gives a boolean `motion_present`.
2. A counter increases while it is true and resets when it is false.
3. When the counter reaches the confirmation count (3), the state becomes Alert and the cooldown timer starts.
4. While motion continues, the timer is refreshed.
5. When no motion has been seen for the cooldown period, the state returns to Clear.

This removes single-frame noise (fewer false alerts) and stops the banner flashing on and off during continuous movement.

The AlertManager exposes an `on_alert` hook. In v1 it updates the overlay and status light. A sound (`winsound.Beep`) can be added there later without touching detection code.

9. Error handling

Failure	Detection	User-facing behaviour
No camera found	<code>isOpened()</code> is false on start	"No camera found. Connect a webcam and try again."
Access denied (Windows privacy setting or another app using it)	Open fails or the first read fails	"Camera access was denied or the camera is in use. Check Windows camera privacy settings and close other apps using the camera."
Camera unplugged mid-run	<code>read()</code> returns <code>ok = False</code> several times in a row	"Camera disconnected." Monitoring stops and the camera is released.
Unexpected exception in worker	Caught at top of worker loop	Generic error shown, camera released, app stays open.
Window closed while running	<code>WM_DELETE_WINDOW</code> handler	Stop event set, thread joined with a timeout, camera released, then exit.

Opening the camera on Windows works best with the DirectShow backend (`cv2.CAP_DSHOW`), with the default backend as fallback.

10. Privacy by design

- Frames are held in memory only: one in the worker, one in the queue, one in the UI. Each is overwritten on the next cycle.
- The code has no calls to `imwrite`, `VideoWriter` or any file API for image data, and no network libraries are used. This can be checked in code review.
- Logging records events and errors only (such as "monitoring started", "camera error"), never image data.
- Stopping releases the camera hardware, so the camera light goes off.

11. Non-functional requirements mapping

Requirement	How the design meets it
Performance	Frames downscaled to 640 px, queue size 1, detection in a separate thread. Target is 15+ processed frames per second.
Accuracy	Blur, MOG2 shadow removal, mask cleaning, minimum area, 3-frame confirmation.
Usability	One window, two buttons, one status line, colour indicator.
Privacy	Section 10.
Reliability	Typed error handling, clean shutdown in <code>finally</code> , daemon worker, no unbounded buffers.
Compatibility	Windows 10/11, any DirectShow-compatible webcam, packaged as a single <code>.exe</code> .
Responsiveness	UI thread only draws; all heavy work is in the worker.

12. Functional requirements traceability

ID	Satisfied by
FR-01	CameraService
FR-02	VideoPanel with queue polling
FR-03	MotionDetector
FR-04	AlertManager and StatusBar
FR-05	OverlayRenderer draws on the frame itself
FR-06, FR-07	Privacy design, section 10
FR-08	Worker loop runs while camera is active
FR-09	Section 9
FR-10	MonitorController.stop() and the stop event

13. Project structure

```
motion_app/
├── main.py                # entry point
├── config.py              # all tunable parameters
├── ui/
│   ├── main_window.py
│   ├── video_panel.py
│   └── status_bar.py
├── core/
│   ├── monitor_controller.py
│   ├── app_state.py
│   ├── camera_service.py
│   ├── motion_detector.py
│   ├── alert_manager.py
│   └── overlay_renderer.py
├── errors.py              # CameraError types
└── tests/
    ├── test_alert_manager.py
    └── test_motion_detector.py
```

14. Technology stack

Purpose	Choice
Language	Python 3.10+
Video capture and processing	opencv-python
Array operations	NumPy
Frame to Tkinter image	Pillow (ImageTk.PhotoImage)
UI	Tkinter
Packaging	PyInstaller (--onefile --windowed)
Testing	pytest

15. Testing strategy

- **Unit tests:** AlertManager with scripted sequences of true/false motion values (confirmation, cooldown, reset). MotionDetector with synthetic frames (static background, moving rectangle, global brightness change).
- **Manual tests:** start and stop repeatedly with no leaks, camera unplugged during run, camera blocked by another app, camera privacy off, long run of 1 hour or more watching memory use.
- **Privacy check:** run with a file monitor and confirm no image or video files are created.

16. Risks and open decisions

Item	Impact	Plan
Client's lighting differs from test setup	False or missed alerts	Expose min area and sensitivity in <code>config.py</code> , tune on site. Consider a sensitivity slider in v2.
Auto-exposure and auto-focus on some webcams cause global changes	False alerts	Warm-up period, shadow handling, optionally ignore frames where more than about 60% of pixels change.
Another app holds the camera	Cannot start	Clear error message, section 9.
Sound alert wanted	Small addition	Use the <code>on_alert</code> hook.
Remote notification wanted (email or SMS)	Breaks offline and no-network assumption	Needs client sign-off and a separate design.

Questions to confirm with the client: Windows only? Built-in or USB camera? Sound alert? Ignore very small movement? All processing kept local?

17. Future extensions

Adjustable sensitivity and region-of-interest selection, sound alerts, a motion-event counter (count only, no images), multiple cameras, and an optional notification channel if the client approves it.